

SelfHealingNN

Implementation Blueprint

A Step-by-Step Guide for Building a
Two-Stage Self-Healing Image Classification System



Tech Stack: PyTorch 2.x • ImageNet-100 • ResNet-18 • ConvVAE

What This Guide Is

This document is an **implementation blueprint**—not a code tutorial. It describes:

- The purpose and responsibility of each file
- What components/functions each file should contain
- How files interact with each other
- Step-by-step build order
- Expected inputs, outputs, and behaviors

For AI models: Use this as a specification to generate the implementation.
For humans: Use this as a roadmap to understand what to build.

Contents

1	Project Overview	3
1.1	Problem Statement	3
1.2	Expected Results	3
1.3	Dataset	3
2	System Architecture	4
2.1	Pipeline Flow	4
2.2	ConvVAE Architecture	4
2.3	Classifier Architecture	4
3	Project File Structure	5
3.1	Complete File List	5
4	File Specifications	6
5	Step-by-Step Build Instructions	11
6	Training Details	13
6.1	VAE Training	13
6.2	Classifier Training	13

7	Expected Outputs	14
7.1	Model Files	14
7.2	Training Metrics	14
7.3	Evaluation Metrics	14
7.4	Visualization Outputs	14
8	Key Implementation Notes	15
9	Quick Reference	16
9.1	Dependencies	16
9.2	Command Summary	16
9.3	File Import Graph	16

1 Project Overview

1.1 Problem Statement

Neural network classifiers trained on clean images suffer severe accuracy degradation when inputs are corrupted by noise (Gaussian noise, salt-and-pepper noise, occlusions, etc.). This project builds a two-stage “self-healing” pipeline that:

1. **Stage A (Healer):** A Convolutional VAE reconstructs clean images from corrupted inputs
2. **Stage B (Classifier):** A pretrained ResNet-18 classifies the healed images

1.2 Expected Results

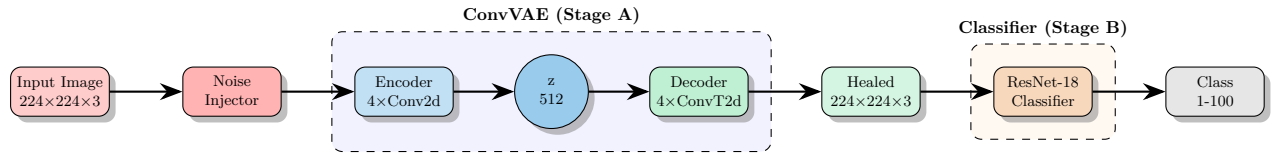
Pipeline	Accuracy	Notes
Clean $\rightarrow$ Classifier	$\sim 94\%$	Baseline
Noisy $\rightarrow$ Classifier	$\sim 52\%$	Degraded
Noisy $\rightarrow$ VAE $\rightarrow$ Classifier	$\sim 88\%$	Recovered

1.3 Dataset

- **Name:** ImageNet-100 (subset of ImageNet with 100 classes)
- **Source:** Kaggle ([ambityga/imagenet100](#))
- **Structure:** Folder per class, JPEG images
- **Image size:** $224 \times 224 \times 3$ (resized)
- **Split:** 80% train, 10% validation, 10% test

2 System Architecture

2.1 Pipeline Flow



2.2 ConvVAE Architecture

Component	Layers	Channel Progression	Spatial Size
Input	—	3	224×224
Encoder Block 1	Conv2d + BN + LeakyReLU	3 → 64	224 → 112
Encoder Block 2	Conv2d + BN + LeakyReLU	64 → 128	112 → 56
Encoder Block 3	Conv2d + BN + LeakyReLU	128 → 256	56 → 28
Encoder Block 4	Conv2d + BN + LeakyReLU	256 → 512	28 → 14
Flatten + FC	Linear	512×14×14 → 512	—
Latent	$\mu, \log \sigma^2$	512 (each)	—
FC + Reshape	Linear	512 → 512×14×14	14×14
Decoder Block 1	ConvT2d + BN + LeakyReLU	512 → 256	14 → 28
Decoder Block 2	ConvT2d + BN + LeakyReLU	256 → 128	28 → 56
Decoder Block 3	ConvT2d + BN + LeakyReLU	128 → 64	56 → 112
Decoder Block 4	ConvT2d + Sigmoid	64 → 3	112 → 224

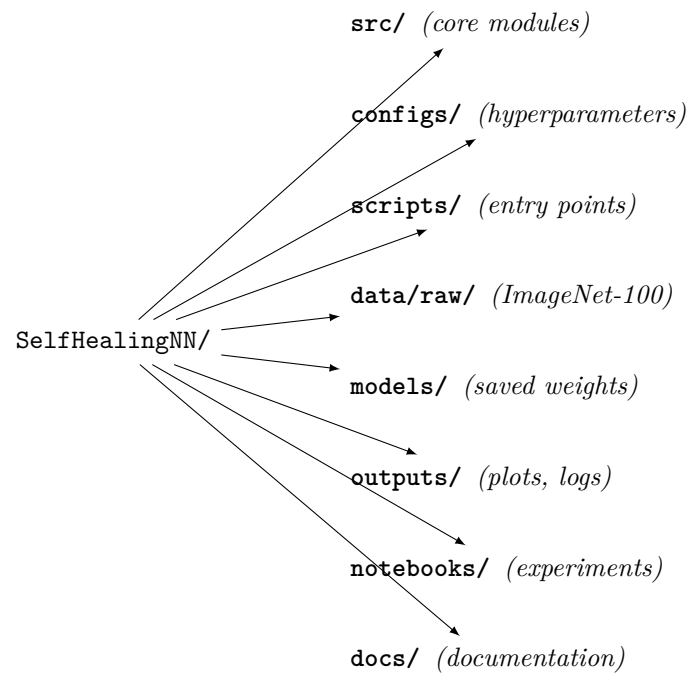
Conv2d parameters: kernel_size=4, stride=2, padding=1 (halves spatial dimensions)

ConvTranspose2d parameters: kernel_size=4, stride=2, padding=1 (doubles spatial dimensions)

2.3 Classifier Architecture

- Use pretrained ResNet-18 from torchvision
- Replace final fully-connected layer: 512 → 100 (number of classes)
- Apply ImageNet normalization (mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])

3 Project File Structure



3.1 Complete File List

File Path	Purpose	Build Order
configs/config.yaml	All hyperparameters	1
src/__init__.py	Package marker (empty)	2
src/dataset.py	Data loading & noise injection	3
src/conv_vae.py	VAE model definition	4
src/classifier.py	ResNet-18 classifier	5
src/train_vae.py	VAE training logic	6
src/train_classifier.py	Classifier training logic	7
src/pipeline.py	End-to-end inference	8
src/evaluate.py	Evaluation metrics	9
scripts/train_vae_runner.py	CLI to train VAE	10
scripts/train_classifier_runner.py	CLI to train classifier	11
scripts/evaluate_runner.py	CLI to run evaluation	12

4 File Specifications

configs/config.yaml

Purpose: Central configuration file for all hyperparameters.

Sections to include:

- **dataset:** root path, image_size (224), num_classes (100), train_split (0.8)
- **noise:** types list, gaussian_std (0.3), salt_pepper_prob (0.1), occlusion_size (32)
- **vae:** latent_dim (512), encoder_channels [64,128,256,512], beta (0.5), learning_rate (3e-4), epochs (50), batch_size (32)
- **classifier:** backbone (resnet18), pretrained (true), learning_rate (1e-4), epochs (30), batch_size (32)
- **training:** device (cuda), num_workers (4), early_stopping_patience (7)

src/dataset.py

Purpose: Data loading, preprocessing, and noise injection.

Components to implement:

1. **Constants:** ImageNet mean [0.485, 0.456, 0.406] and std [0.229, 0.224, 0.225]
2. **Class NoiseInjector:** Methods for adding noise to tensors
 - add_gaussian_noise(image, std) — add Gaussian noise, clamp to valid range
 - add_salt_pepper(image, prob) — randomly set pixels to min/max values
 - add_occlusion(image, patch_size) — black out random rectangular patch
3. **Class ImageNet100Dataset:** Wraps torchvision.datasets.ImageFolder
 - Transform pipeline: Resize → CenterCrop → ToTensor → Normalize
 - Returns (image, label) tuples
4. **Class NoisyImageNet100Dataset:** Wraps base dataset, adds noise
 - Constructor takes: base_dataset, noise_type, noise_params, return_clean flag
 - Returns (noisy_image, clean_image, label) for VAE training
 - Returns (noisy_image, label) for classifier testing
5. **Function get_dataloaders:** Factory function
 - Splits dataset into train/val/test (80/10/10)
 - Returns three DataLoader objects
 - Accepts flags for noisy mode and noise parameters

`src/conv_vae.py`

Purpose: Convolutional Variational Autoencoder model.

Components to implement:

1. **Class ConvEncoder:**

- 4 convolutional blocks (Conv2d + BatchNorm + LeakyReLU)
- Channel progression: $3 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 512$
- Flatten output and project to μ and $\log \sigma^2$ vectors (latent_dim each)
- Forward returns: (mu, logvar)

2. **Class ConvDecoder:**

- FC layer from latent_dim to $512 \times 14 \times 14$
- 4 transposed conv blocks (ConvTranspose2d + BatchNorm + LeakyReLU)
- Final layer uses Sigmoid activation (output in $[0,1]$)
- Forward returns: reconstructed image

3. **Class ConvVAE:**

- Contains encoder and decoder
- Implements reparameterization trick: $z = \mu + \sigma \cdot \epsilon$
- Forward returns: (reconstruction, mu, logvar)

4. **Function vae_loss:**

- MSE reconstruction loss between output and target
- KL divergence: $-0.5 \times \text{mean}(1 + \log \sigma^2 - \mu^2 - \sigma^2)$
- Total loss = recon_loss + $\beta \times \text{kl_loss}$
- Returns: (total_loss, dict with component losses)

`src/classifier.py`

Purpose: Image classifier using pretrained ResNet-18.

Components to implement:

1. **Class SelfHealingClassifier:**

- Load ResNet-18 with ImageNet pretrained weights
- Replace final FC layer: $512 \rightarrow \text{num_classes}$ (100)
- Forward returns: logits (before softmax)

2. **Function get_classifier:** Factory function with num_classes and pretrained args

`src/train_vae.py`

Purpose: Training loop for the VAE.

Components to implement:

1. **Function train_vae:**

- Arguments: model, train_loader, val_loader, device, epochs, lr, beta, save_path
- Optimizer: AdamW
- Scheduler: CosineAnnealingLR
- Training loop:
 - For each batch: get (noisy, clean, label)
 - Forward pass: recon, mu, logvar = model(noisy)
 - Compute loss against clean target
 - Backward and step
- Validation loop: compute average validation loss (no gradients)
- Save best model based on validation loss
- Return training history dictionary

2. **Optional - Class EarlyStopping:** Stop if val loss doesn't improve for N epochs

`src/train_classifier.py`

Purpose: Training loop for the classifier.

Components to implement:

1. **Function train_classifier:**

- Arguments: model, train_loader, val_loader, device, epochs, lr, save_path
- Loss function: CrossEntropyLoss
- Optimizer: AdamW
- **Two-phase training strategy:**
 - Phase 1 (5 epochs): Freeze all backbone layers, train only final FC
 - Phase 2 (remaining epochs): Unfreeze all, fine-tune entire network with lower LR
- Validation: compute accuracy
- Save best model based on validation accuracy
- Return training history dictionary

src/pipeline.py

Purpose: End-to-end inference combining VAE and classifier.

Components to implement:

1. **Class SelfHealingPipeline:**

- Constructor takes: vae model, classifier model
- `heal(noisy_image)` method: pass through VAE, return reconstruction
- `forward(noisy_image)` method: heal then classify
 - Returns: (predicted_class, confidence, healed_image)
- `save(path)` method: save both model state dicts to single file
- `load(path, device)` method: load state dicts into models

2. All inference methods should use `@torch.no_grad()` decorator

src/evaluate.py

Purpose: Evaluation metrics and comparison functions.

Components to implement:

1. **Function evaluate_classifier:**

- Compute accuracy, top-5 accuracy, confusion matrix
- Arguments: model, dataloader, device
- Returns: dict with metrics

2. **Function evaluate_pipeline:**

- Run full pipeline (noisy $\rightarrow$ VAE $\rightarrow$ classifier)
- Compare against baseline (noisy $\rightarrow$ classifier only)
- Returns: dict with both accuracy sets

3. **Function compute_reconstruction_metrics:**

- PSNR (Peak Signal-to-Noise Ratio)
- SSIM (Structural Similarity Index) - use scikit-image
- Arguments: original_images, reconstructed_images

4. **Function evaluate_noise_robustness:**

- Test at multiple noise levels (0.1, 0.2, 0.3, 0.5, 0.7)
- Returns: table of accuracies per noise level

scripts/train_vae_runner.py

Purpose: Command-line entry point for VAE training.

Implementation:

1. Add parent directory to sys.path for imports
2. Load config.yaml
3. Create noisy dataloaders (return_clean=True)
4. Instantiate ConvVAE model
5. Call train_vae function
6. Optionally save training plots to outputs/plots/

scripts/train_classifier_runner.py

Purpose: Command-line entry point for classifier training.

Implementation:

1. Load config.yaml
2. Create clean dataloaders (no noise)
3. Instantiate SelfHealingClassifier
4. Call train_classifier function
5. Save training plots

scripts/evaluate_runner.py

Purpose: Command-line entry point for evaluation.

Implementation:

1. Load trained VAE and classifier from models/ directory
2. Create SelfHealingPipeline
3. Run evaluate_pipeline on test set
4. Run evaluate_noise_robustness at multiple noise levels
5. Print results table and save to outputs/

5 Step-by-Step Build Instructions

Phase 1: Environment Setup

1. Create project directory structure as shown in Section 3
2. Create Python virtual environment
3. Install dependencies: torch, torchvision, numpy, matplotlib, scikit-image, scikit-learn, pyyaml, tqdm, pandas
4. Verify CUDA availability
5. Create `configs/config.yaml` with all parameters

Phase 2: Dataset Setup

1. Download ImageNet-100 from Kaggle
2. Extract to `data/raw/train/` (should have 100 subdirectories)
3. Verify structure: each class folder contains JPEG images
4. Implement `src/dataset.py`
5. Test: create dataloaders, visualize a few samples with noise

Phase 3: Model Implementation

1. Implement `src/conv_vae.py`
 - Test: create model, pass random tensor, verify output shapes
 - Encoder output: (batch, 512), (batch, 512) for mu/logvar
 - Decoder output: (batch, 3, 224, 224)
2. Implement `src/classifier.py`
 - Test: verify ResNet loads, final layer outputs 100 classes
3. Implement `src/pipeline.py`
 - Test: combine untrained models, verify forward pass works

Phase 4: Training Implementation

1. Implement `src/train_vae.py`
2. Implement `src/train_classifier.py`
3. Create runner scripts in `scripts/`
4. Test with 1-2 epochs to verify everything works

Phase 5: Full Training

1. Train VAE first:

- Run: `python scripts/train_vae_runner.py`
- Expected time: ~4-6 hours on RTX 3050
- Output: `models/conv_vae_best.pth` (~200 MB)
- Monitor: validation loss should decrease steadily

2. Train classifier second:

- Run: `python scripts/train_classifier_runner.py`
- Expected time: ~2-3 hours on RTX 3050
- Output: `models/resnet_classifier.pth` (~45 MB)
- Monitor: validation accuracy should reach ~90%+

Phase 6: Evaluation

1. Implement `src/evaluate.py`
2. Create `scripts/evaluate_runner.py`
3. Run evaluation: `python scripts/evaluate_runner.py`
4. Generate comparison table across noise levels
5. Visualize: noisy → healed → prediction examples

6 Training Details

6.1 VAE Training

Input: Noisy image (corrupted)

Target: Clean image (original)

Loss: $\text{MSE}(\text{reconstruction, clean}) + \beta \times \text{KL_divergence}$

Key insight: VAE learns to remove noise by reconstructing the clean version

Hyperparameter	Value
Optimizer	AdamW
Learning rate	3e-4
Scheduler	CosineAnnealingLR
Batch size	32
Epochs	50
β (KL weight)	0.5
Latent dimension	512

6.2 Classifier Training

Input: Clean image

Target: Class label (0-99)

Loss: CrossEntropyLoss

Key insight: Train on clean images; during inference, VAE provides “clean-like” inputs

Hyperparameter	Value
Pretrained weights	ImageNet
Phase 1 (freeze backbone)	5 epochs
Phase 2 (fine-tune all)	25 epochs
Learning rate	1e-4
Batch size	32

7 Expected Outputs

7.1 Model Files

File	Size	Contents
models/conv_vae_best.pth	~200 MB	VAE encoder + decoder weights
models/resnet_classifier.pth	~45 MB	Fine-tuned ResNet-18 weights

7.2 Training Metrics

- VAE: final validation loss ~0.02–0.03
- Classifier: final validation accuracy ~92–95%

7.3 Evaluation Metrics

Noise Level (σ)	Noisy→Classifier	Noisy→VAE→Classifier	Recovery
0.1	~82%	~93%	+11%
0.2	~68%	~91%	+23%
0.3	~52%	~88%	+36%
0.5	~32%	~78%	+46%
0.7	~19%	~64%	+45%

7.4 Visualization Outputs

- outputs/plots/vae_loss_curve.png — training/validation loss over epochs
- outputs/plots/classifier_accuracy.png — accuracy over epochs
- outputs/plots/reconstruction_samples.png — noisy vs. healed comparisons
- outputs/plots/noise_robustness.png — accuracy vs. noise level graph

8 Key Implementation Notes

Critical Details

1. **Normalization consistency:** Always apply ImageNet normalization (mean/std) to images. Both VAE and classifier must use the same normalization.
2. **Noise clamping:** After adding noise, clamp values to valid range (approximately -3 to +3 for normalized images) to prevent extreme outliers.
3. **VAE output activation:** Use Sigmoid on decoder output, then denormalize, or match the input range. Ensure reconstruction target matches output range.
4. **Random seed:** Set seed for dataset splitting (42) to ensure reproducible train/val/test splits.
5. **Two-phase classifier training:** Freezing the backbone initially prevents catastrophic forgetting of pretrained features.
6. **Model saving:** Save `state_dict` (not entire model) for compatibility.
7. **Inference mode:** Use `model.eval()` and `torch.no_grad()` during evaluation.
8. **Device handling:** Move both model and data to same device (cuda/cpu).

Common Pitfalls to Avoid

- Not matching normalization between training and inference
- Forgetting to call `model.eval()` during validation
- Training VAE on clean→clean instead of noisy→clean
- Using reconstruction loss against noisy target (should be clean)
- Not unfreezing classifier backbone in phase 2
- Memory issues: reduce batch size if OOM

9 Quick Reference

9.1 Dependencies

Package	Purpose
torch, torchvision	Deep learning framework
numpy	Numerical operations
matplotlib	Plotting
scikit-image	SSIM metric
scikit-learn	Train/test split utilities
pyyaml	Config file parsing
tqdm	Progress bars
pandas	Results tables

9.2 Command Summary

<code>python scripts/train_vae_runner.py</code>	Train the VAE healer
<code>python scripts/train_classifier_runner.py</code>	Train the classifier
<code>python scripts/evaluate_runner.py</code>	Run evaluation

9.3 File Import Graph

